

```

[87]: from sklearn.datasets import fetch_openml

[88]: dataset = fetch_openml('mnist_784', as_frame=False)

[89]: dataset.keys()

[89]: dict_keys(['data', 'target', 'frame', 'categories', 'feature_names', 'target_names', 'DESCR', 'details', 'url'])

[90]: dataset.DESCR

[90]: """Author*: Yann LeCun, Corinna Cortes, Christopher J.C. Burges \n**Source*: [MNIST Website](http://yann.lecun.com/exdb/mnist)
ease cite*: \n\nThe MNIST database of handwritten digits with 784 features, raw data available at: http://yann.lecun.com/exdb/mnist
n a training set of the first 60,000 examples, and a test set of 10,000 examples \n\nIt is a subset of a larger set available at: http://yann.lecun.com/exdb/mnist
been size-normalized and centered in a fixed-size image. It is a good database for people who want to try learning techniques
hods on real-world data while spending minimal efforts on preprocessing and formatting. The original black and white (bilevel)
normalized to fit in a 28x28 pixel box while preserving their aspect ratio. The resulting images contain grey levels as a result of the
hnique used by the normalization algorithm. the images were centered in a 28x28 image by computing the center of mass of the image so as to position this point at the center of the 28x28 field. \n\nWith some classification methods (particularly temp
SVM and K-nearest neighbors), the error rate improves when the digits are centered by bounding box rather than center of mass. \n\nIf you are using this database for machine learning, you should report it in your publications. The MNIST database was constructed from NIST's NIST originally design
"""

```

```
[91]: X,Y = dataset.data,dataset.target
```

```
[92]: type(X)
```

```
[92]: numpy.ndarray
```

```
[93]: type(Y)
```

```
[93]: numpy.ndarray
```

```
[94]: X.shape
```

```
[94]: (70000, 784)
```

```
[95]: Y[0]
```

```
[95]: '5'
```

```
[96]: type(Y[0])
```

```
[96]: str
```

```
[97]: Y = Y.astype(int)
```

```
[98]: Y[0]
```

```
[98]: np.int64(5)
```

```
[99]: Y
```

```
[99]: array([5, 0, 4, ..., 4, 5, 6])
```

```
[100]: Y.shape
```

```
[100]: (70000,)
```

```
[101]: Y_train = Y[:60000]
       Y_test = Y[60000:]
```

```
[102]: X_train = X[:60000]
       X_test = X[60000:]
```

```
[103]: array([[ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  3, 18, 18, 18,
126, 136, 175, 26, 166, 255, 247, 127,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0, 30, 36, 94, 154, 170, 253,
253, 253, 253, 253, 225, 172, 253, 242, 195, 64,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0, 49, 238, 253, 253, 253,
253, 253, 253, 253, 253, 251, 93, 82, 82, 56, 39,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 18, 219, 253,
253, 253, 253, 253, 198, 182, 247, 241,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 80, 156, 107, 253, 253, 205, 11,  0, 43, 154,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,
  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,
  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,
  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -,  -])
```

```
[104]: X.shape
```

```
[104]: (70000, 784)
```

BINARY CLASSIFICATION

```
[105]: import numpy as np
```

```
[106]: np.unique_values(Y)
```

```
[106]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
[107]: Y_BIN = (Y==5)
```

```
[111]: Y_BIN
```

```
[111]: array([ True, False, False, ...,  True, False])
```

```
[112]: Y_BIN_train = Y_BIN[:60000]  
Y_BIN_test = Y_BIN[60000:]
```

```
[113]: Y_BIN_train
```

```
[113]: array([ True, False, False, ...,  True, False, False])
```

```
[114]: from sklearn.linear_model import SGDClassifier
```

```
[115]: model = SGDClassifier(random_state = 42)
```

```
[116]: model.fit(X_train,Y_BIN_train)
```

```
[116]: SGDClassifier
SGDClassifier(random_state=42)
```

```
[117]: model.predict(X_train[0].reshape(1, -1))
```

```
[117]: array([ True])
```

```
[118]: X_train[0].shape
```

```
[118]: (784,)
```

```
[119]: pred = model.predict(X_train)
```

```
[120]: pred
```

```
[120]: array([ True, False, False, ...,  True, False, False])
```

```
[121]: pred[:20]
```

```
[121]: array([ True, False, False, False, False, False, False, False, False,
        False, False,  True, False, False, False, False, False, False,
        False, False])
```

```
[122]: Y[:20]
```

```
[122]: array([5, 0, 4, 1, 9, 2, 1, 3, 1, 4, 3, 5, 3, 6, 1, 7, 2, 8, 6, 9])
```

```
[123]: k = model.predict([X_train[0]])
```

```
[123]: k=model.predict([X_train[0]])

[124]: Y_train[:10]

[124]: array([5, 0, 4, 1, 9, 2, 1, 3, 1, 4])

[125]: from sklearn.metrics import confusion_matrix

[126]: confusion_matrix(Y_BIN_train,pred)

[126]: array([[52316, 2263],
             [ 601, 4820]])

[127]: confusion_matrix(pred,Y_BIN_train)

[127]: array([[52316, 601],
             [ 2263, 4820]])

[128]: from sklearn.metrics import precision_score,recall_score,accuracy_score,f1_score

[129]: precision_score(Y_BIN_train,pred)

[129]: 0.6805026118876183

[130]: recall_score(Y_BIN_train,pred)

[130]: 0.8891348459693783

[131]: accuracy_score(Y_BIN_train,pred)

[131]: 0.9522666666666667

[132]: f1_score(Y_BIN_train,pred)
```

```
[132]: f1_score(Y_BIN_train,pred)
```

```
[132]: 0.7709532949456174
```

```
[133]: precision_score(pred,Y_BIN_train)
```

```
[133]: 0.8891348459693783
```

Logistic Regression:

```
[136]: x = np.linspace(-6, 6, num=1000)
plt.figure(figsize=(10, 6))
plt.plot(x, (1 / (1 + np.exp(-x))))
plt.xlabel("x")
plt.ylabel("y")
plt.title("Sigmoid Function")
```

```
[136]: Text(0.5, 1.0, 'Sigmoid Function')
```